



Reporte Técnico de Actividades Práctico- Experimentales Nro. 2

1. Datos de Identificación del Estudiante y la Práctica

Nombre de los estudiante(s)	Tyrone Encarnación Erick Andrade Jhoan Rojas Matías Ortega
Asignatura	Matemáticas Discretas
Ciclo	1A
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 FASE 2
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad
Nombre del Docente	Ing. Mario Cueva
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 2: Construcción de Tablas de Verdad y Operaciones Combinadas (Semana 2: 13-17 abril 2026)

1. El docente profundiza en la construcción de tablas de verdad para proposiciones compuestas con múltiples conectores, explicando el orden de evaluación (jerarquía de operadores) y el método sistemático para generar todas las combinaciones posibles de valores de verdad.
2. Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales, identificando los casos en que la expresión resulta verdadera, falsa o es una tautología/contradicción/contingencia.
3. Práctica grupal: cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).
4. Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en Python/C/Java).
5. Entrega del primer portafolio de ejercicios individuales (propuestas y tablas de verdad) a través de la plataforma EVA. Discusión sobre la ética en el uso de la lógica para la toma de decisiones algorítmicas y su impacto en la sociedad.



5. Resultados

Matemáticas Discretas APE1 FASE 2

1. Ejercicios realizados en clase

Ejercicio 1

Premisas Iniciales:

$$p \vee q = P_1$$

$$\neg r = P_2$$

$$Q \rightarrow r = P_3$$

Desarrollo de Inferencias:

Premisa 4: $\neg q$

Premisa 4 obtenida mediante Modus Tollendo Tollens entre la premisa 2 ($\neg r$) y la premisa 3 ($q \rightarrow r$).

Premisa 5: p

Premisa 5 obtenida mediante Modus Tollendo Ponens entre la premisa 1 ($p \vee q$) y la premisa 4 ($\neg q$).

Ejercicio 2

$p \wedge p \leftrightarrow p$	
Simplificación	Leyes usadas
$p \rightarrow p \wedge (p \vee \neg p)$	Ley de Complemento
$p \rightarrow (p \wedge p) \vee (p \wedge \neg p)$	Ley distributiva
$p \rightarrow (p \wedge p) \vee (F)$	Ley de Complemento
$p \rightarrow p \wedge p$	Ley de Idempotencia
$p \rightarrow p$	Ley de Identidad



2. Simplificación de expresiones lógicas

Ejercicio 1

$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)] \equiv (q \vee r)$	
Simplificación	Leyes usadas
$[(p \vee q \vee r) \vee \neg (p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$	Ley de Morgan
$V \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$	Ley de Complemento
$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)$	Ley de identidad
$[q \wedge (r \vee \neg r)] \vee [\neg q \vee (\neg q \wedge r)]$	Ley distributiva
$(q \wedge V) \vee (\neg q \wedge r)$	Ley del complemento
$q \vee (\neg \vee r)$	Ley de identidad
$(q \wedge \neg q) \wedge (q \vee r)$	Ley distributiva
$V \wedge (q \vee r)$	Ley del complemento
$q \vee r$	Ley de identidad

Ejercicio 2

$\neg (p \wedge q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \wedge r)]$	
Simplificación	Leyes usadas
$(\neg p \vee \neg q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee r)]$	Ley de Morgan
$(\neg p \vee \neg q) \vee [(p \vee (q \wedge \neg q)) \wedge (\neg p \vee r)]$	Ley Distributiva
$(\neg p \vee \neg q) \vee [(p \vee F) \wedge (\neg p \vee r)]$	Ley de Complemento
$(\neg p \vee \neg q) \vee [p \wedge (\neg p \vee r)]$	Ley de Identidad
$(\neg p \vee \neg q) \vee [(p \wedge \neg p) \vee (p \wedge r)]$	Ley Distributiva
$(\neg p \vee \neg q) \vee [F \vee (p \wedge r)]$	Ley de Complemento
$(\neg p \vee \neg q) \vee (p \vee r)$	Ley de Identidad
$(\neg p \vee (p \wedge r)) \vee (\neg q \vee (p \wedge r))$	Ley Asociativa + Conmutativa
$(\neg p \vee r) \vee (\neg q \vee (p \wedge r))$	Ley de Absorción
$(\neg p \vee r) \vee \neg q$	Ley de Absorción
$\neg p \vee \neg q \vee r$	Ley Asociativa + Conmutativa



Ejercicio 3

$[((\neg p \vee \neg q \vee \neg r) \vee (p \wedge q \wedge r)) \wedge ((p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r))]$	
Simplificación	Leyes usadas
$[\neg (p \wedge q \vee r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \wedge r)]$	Ley de Morgan
$\vee \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley de Complemento
$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \wedge r)]$	Ley de Identidad
$[p \wedge (r \vee \neg r)] \vee (\neg p \wedge r)$	Ley distributiva
$(p \wedge \vee) \vee (\neg p \wedge r)$	Ley de complemento
$p \vee (\neg p \wedge r)$	Ley de identidad
$(p \vee \neg p) \wedge (p \vee r)$	Ley distributiva
$\vee \wedge (p \vee r)$	Ley del complemento
$p \vee r$	Ley de identidad

3. Cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación.

3.1 If-else

Las expresiones booleanas se usan en la estructura if-else para decidir qué instrucciones ejecutar según una condición. En un if, el programa evalúa una expresión que da verdadero o falso; si es verdadera, ejecuta un bloque de código, y si es falsa, pasa al else, donde se ejecuta otro bloque alternativo. Estas condiciones pueden incluir operadores lógicos como AND, OR y NOT para combinar varios criterios. Así, el if-else permite que el programa tome decisiones automáticamente según los valores de las variables.



Ejemplo:

Problema 1: Programa de aprobación de estudiantes

Se desea desarrollar un programa que determine si un estudiante **aprueba o reprueba** una materia según las siguientes condiciones:

- El estudiante **aprueba** si:
 - Tiene una **nota alta** (mayor o igual a 7),
O
 - Tiene una **nota media** (mayor o igual a 5) y **además** entregó el proyecto final.

En caso contrario, el estudiante **reprueba**.

Definición de variables

- **p**: el estudiante tiene $\text{nota} \geq 7$ (nota alta)
- **r**: el estudiante tiene $\text{nota} \geq 5$ (nota media)
- **q**: el estudiante entregó el proyecto

Expresión lógica del problema

$$p \vee (r \wedge q)$$



Tabla de verdad

p	r	q	$r \wedge q$	$p \vee (r \wedge q)$	Resultado
V	V	V	V	V	Aprueba
V	V	F	F	V	Aprueba
V	F	V	F	V	Aprueba
V	F	F	F	V	Aprueba
F	V	V	V	V	Aprueba
F	V	F	F	F	Reprueba
F	F	V	F	F	Reprueba
F	F	F	F	F	Reprueba



Algoritmo en Java

```
import java.util.Scanner;

public class AprobacionEstudiante { public
    static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese la nota: "); int
        nota = sc.nextInt();

        System.out.print("¿Entregó el proyecto? (true/false): "); boolean q =
        sc.nextBoolean();

        boolean p = (nota >= 7); // nota alta
        boolean r = (nota >= 5); // nota media

        if (p || (r && q)) {
            System.out.println("Aprueba");
        } else {
            System.out.println("Reprueba");
        }

        sc.close();
    }
}
```



Ejemplo de ejecución

Caso 1

- Nota = 6
- Proyecto = entregado

→ $p = F, r = V, q = V$

→ Aprueba

Caso 2

- Nota = 4
- Proyecto = no entregado

→ $p = F, r = F, q = F$

→ Resultado = F

El programa muestra:

Reprueba



Problema 2: Sistema de acceso a una plataforma

Se desea desarrollar un programa que determine si un usuario puede **acceder o no** a una plataforma, según las siguientes condiciones:

- El usuario puede acceder si:
 - Tiene la **contraseña correcta**, Y además
 - Tiene activada la **verificación en dos pasos** O
 - Está usando un **dispositivo confiable**. En

caso contrario, el acceso será denegado.

Definición de variables

- **p**: contraseña correcta
- **q**: verificación en dos pasos activa
- **r**: dispositivo confiable

Expresión lógica del problema

$$p \wedge (q \vee r)$$



Tabla de verdad

p	q	r	$q \vee r$	$p \wedge (q \vee r)$	Resultado
V	V	V	V	V	Acceso permitido
V	V	F	V	V	Acceso permitido
V	F	V	V	V	Acceso permitido
V	F	F	F	F	Acceso denegado
F	V	V	V	F	Acceso denegado
F	V	F	V	F	Acceso denegado
F	F	V	V	F	Acceso denegado
F	F	F	F	F	Acceso denegado



Algoritmo en Java:

```
import java.util.Scanner;
```

```
public class AccesoPlataforma {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("¿Contraseña correcta? (true/false): "); boolean p =
```

```
        sc.nextBoolean();
```

```
        System.out.print("¿Verificación en dos pasos activa? (true/false): "); boolean q =
```

```
        sc.nextBoolean();
```

```
        System.out.print("¿Dispositivo confiable? (true/false): "); boolean r =
```

```
        sc.nextBoolean();
```

```
        if (p && (q || r)) { System.out.println("Acceso  
            permitido");
```

```
        } else {
```

```
            System.out.println("Acceso denegado");
```

```
        }
```

```
        sc.close();
```

```
    }
```

```
}
```

Ejemplo de ejecución

Caso 1

- Contraseña correcta = verdadero
- Verificación = falso
- Dispositivo confiable = verdadero

→ $(q \text{ OR } r) = V$

→ $p \text{ AND resultado} = V \text{ El}$

programa muestra:

Acceso permitido

Caso 2

- Contraseña correcta = verdadero
- Verificación = falso
- Dispositivo confiable = falso

→ $(q \text{ OR } r) = F$

→ $p \text{ AND resultado} = F \text{ El}$

programa muestra:

Acceso denegado

Caso 3

- Contraseña incorrecta = falso No importa lo demás

→ $p = F$

→ Resultado final = F El

programa muestra:

Acceso denegado

3.2 While

Las expresiones booleanas son fundamentales en la evaluación de condiciones en estructuras repetitivas como while en lenguajes como Python, C y Java, ya que determinan si el ciclo continúa ejecutándose o se detiene. Una expresión booleana es aquella que produce un valor verdadero o falso mediante el uso de operadores relacionales. En un ciclo while, esta expresión se evalúa antes de cada iteración: si es verdadera, el bloque de código se ejecuta, y si es falsa, el ciclo termina. Aunque la lógica es la misma en los tres lenguajes, en C se permite que valores numéricos representen verdadero o falso (donde 0 es falso y cualquier otro valor es verdadero), mientras que en Python y Java la condición debe evaluarse directamente como un valor booleano.

Ejemplos:

Problema 1: Control de Temperatura de un Reactor

Se requiere un sistema que mantenga un reactor funcionando mientras las condiciones de seguridad se cumplan. El reactor se detiene (sale del while) si la situación se vuelve peligrosa.

Condiciones para que el reactor siga operando:

- La temperatura es menor a 100°C (p).
- Y (el sistema de enfriamiento está activo (q) O el nivel de presión es normal (r)).

1. Definición de variables

- p: La temperatura es < 100 (Seguro).
- q: El enfriamiento está encendido.
- r: La presión es normal.

2. Expresión lógica del problema (Condición de continuación)

$$p \wedge (q \vee r)$$



Tabla de verdad

p	q	r	$(q \vee r)$	$p \wedge (q \vee r)$	Estado del Ciclo
V	V	V	V	V	CONTINÚA
V	V	F	V	V	CONTINÚA
V	F	V	V	V	CONTINÚA
V	F	F	F	F	TERMINA
F	V	V	V	F	TERMINA
F	V	F	V	F	TERMINA
F	F	V	V	F	TERMINA
F	F	F	F	F	TERMINA



3. Algoritmo en Java

```
import java.util.Scanner;

public class ControlReactor {

    public static void main(String[] args)

        { Scanner sc = new

        Scanner(System.in);

        //Estado inicial para entrar al

        ciclo boolean enfriamiento

        = true;

        boolean presionNormal =

        true; int temperatura = 20;

        while (temperatura < 100 CC (enfriamiento || presionNormal)) {

            System.out.println("Reactor estable. Temperatura actual: " + temperatura + "°C");

            System.out.print("Ingrese nueva temperatura: "); temperatura

            = sc.nextInt();

            System.out.print("¿Enfriamiento activo? (true/false): ");

            enfriamiento = sc.nextBoolean();

            System.out.print("¿Presion normal? (true/false): ");

            presionNormal = sc.nextBoolean();

        }

        System.out.println("ALERTA: Condiciones inseguras detectadas. Ciclo terminado."); sc.close();

    }
```



Problema 2: Acceso a una Bóveda Bancaria

Un sistema de seguridad permite seguir intentando procesos de validación mientras el usuario no exceda los intentos y mantenga credenciales válidas.

Condiciones para seguir en el proceso de validación:

- El número de intentos es menor a 3 (p).
- Y NO se ha activado la alarma de intrusión ($\neg q$).

Definición de variables

- p: Intentos < 3 .
- q: Alarma activada.

Expresión lógica del problema

$$p \wedge \neg q$$

Tabla de verdad

p (Intentos < 3)	q (Alarma)	$\neg q$	$p \wedge \neg q$	Estado del Ciclo
V	V	F	F	TERMINA
V	F	V	V	CONTINÚA
F	V	F	F	TERMINA
F	F	V	F	TERMINA



3. Algoritmo en Java

```
import java.util.Scanner;
```

```
public class AccesoBodega {
```

```
    public static void main(String[] args)
```

```
    { Scanner sc = new
```

```
    Scanner(System.in); int intentos =
```

```
    0;
```

```
    boolean alarma = false;
```

```
    // El ciclo se ejecuta mientras p sea verdadero Y q sea falso while
```

```
    (intentos < 3 CC !alarma) {
```

```
        System.out.println("--- Intento #" + (intentos + 1) + " ---");
```

```
        System.out.print("¿Se ha activado la alarma por movimiento? (true/false): "); alarma =
```

```
        sc.nextBoolean();
```

```
        if(!alarma) {
```

```
            System.out.println("Escaneando huella... Acceso denegado temporalmente."); intentos++;
```

```
        }
```

```
    }
```

```
    if(alarma) {
```

```
        System.out.println("BLOQUEO TOTAL: Alarma activada.");
```

```
    } else {
```

```
        System.out.println("Límite de intentos alcanzado. Contacte al administrador.");
```

```
    }
```

```
    sc.close();
```

```
}
```

```
}
```



3.3 Operadores lógicos en Python/C/Java

Las expresiones booleanas en programación equivalen a proposiciones compuestas evaluadas mediante tablas de verdad, donde la conjunción lógica ($\wedge$), que exige que todas las premisas sean verdaderas, se implementa con `&&` en C y Java y con `and` en Python; la disyunción inclusiva ($\vee$), que resulta verdadera si al menos una condición se cumple, se representa con `||` en C y Java y con `or` en Python; y la negación ($\neg$), un operador unario que invierte el valor lógico, utiliza `!` en C y Java y `not` en Python. Para optimizar estas evaluaciones, los tres lenguajes aplican un principio de inferencia lógica conocido como evaluación en cortocircuito, el cual deduce el resultado global desde la primera proposición analizada: si el primer término de una conjunción es falso, o el de una disyunción es verdadero, el sistema omite la evaluación del resto de la expresión, ya que el valor de verdad final ha quedado lógicamente determinado sin necesidad de deducciones adicionales.

Ejemplos:

Problema 1: Acceso a Beca Universitaria

Premisas:

p: El promedio es mayor a 9.0.

q: El estudiante tiene excelencia deportiva.

Condición lógica: Se obtiene la beca si ocurre p O q. (Disyunción: $p \vee q$)

Tabla de Verdad ($p \vee q$)

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F



Código en Python

```
Promedio_alto = True # p
```

```
Excelencia_deportiva = False
```

```
# q
```

```
If promedio_alto or excelencia_deportiva:
```

```
    Print("Beca concedida.")
```

```
Else:
```

```
    Print("No cumple los requisitos para la beca.")
```

Problema 2: Validación de Transacción Bancaria

Premisas:

p: El PIN es correcto.

q: Hay fondos suficientes.

r: El cajero tiene efectivo.

Condición lógica: La transacción es exitosa si $(p \vee q) \wedge r$. (Conjunción: $(p \wedge q) \wedge r$)

Tabla de Verdad $(p \wedge q) \wedge r$

p	q	r	$(p \wedge q)$	$(p \wedge q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	F	F
V	F	F	F	F
F	V	V	F	F
F	V	F	F	F
F	F	V	F	F
F	F	F	F	F



unl

Universidad
Nacional
de Loja

Código Java

```
Boolean pinCorrecto = true; // p
```

```
Boolean fondosSuficientes = true; //
```

```
q Boolean cajeroConEfectivo =
```

```
false; // r
```

```
If (pinCorrecto CC fondosSuficientes CC cajeroConEfectivo) {  
    System.out.println("Transacción exitosa. Retire su dinero.");  
} else {  
    System.out.println("Error en la transacción.");  
}
```

4. Links del portafolio en drive de los integrantes del grupo

4.1 Tyrone Encarnación

<https://drive.google.com/drive/folders/1WsFla--QkQ5Ywd0llmL-k8oIla7CzJkq?usp=sharing>

4.2 Erick Andrade

[https://drive.google.com/drive/folders/1KUsk7Nm7JO2qNrXFjRho6i0qSWbntOXK?usp=drive link](https://drive.google.com/drive/folders/1KUsk7Nm7JO2qNrXFjRho6i0qSWbntOXK?usp=drive_link)

4.3 Matias Ortega

https://drive.google.com/drive/folders/1AvS_OQAKhYvWEwmBj4_iMgUa0mfNaVlf?usp=sharing

4.4 Jhoan Rojas

[https://drive.google.com/drive/folders/1_WE3EuplR2TAZ8h7NLoBuyABo11-O1hm?usp=drive link](https://drive.google.com/drive/folders/1_WE3EuplR2TAZ8h7NLoBuyABo11-O1hm?usp=drive_link)

6. Preguntas de Control

1. Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación.

Proposiciones simples y compuestas: conceptos fundamentales

Una **proposición simple** (también llamada proposición atómica) es un enunciado declarativo que tiene un valor de verdad único y bien definido —verdadero (V) o falso (F)— y que no puede descomponerse en proposiciones más elementales. Se representa usualmente con letras minúsculas como p, q, r. Ejemplos:

- p: "El número 7 es primo." (Verdadero)
- q: "Python es un lenguaje de bajo nivel." (Falso)

Una **proposición compuesta** se forma combinando dos o más proposiciones simples mediante conectores lógicos: negación ($\neg$), conjunción ($\wedge$), disyunción ($\vee$), condicional ($\rightarrow$) y bicondicional ($\leftrightarrow$). Su valor de verdad depende del valor de sus componentes y del conector utilizado. Ejemplos:

- $p \wedge q$: "El número 7 es primo Y Python es un lenguaje de bajo nivel." (Falso, pues q es F)
- $p \rightarrow q$: "Si el número 7 es primo, entonces Python es de bajo nivel." (Falso)

Diferencias clave

Característica	Proposición Simple	Proposición Compuesta
Estructura	Atómica, indivisible	Formada por 2 o más simples
Valor de verdad	Independiente	Depende de sus componentes
Conectores	No posee	$\neg, \wedge, \vee, \rightarrow, \leftrightarrow$
Tabla de verdad	2 filas (V/F)	2^n filas (n variables)



Importancia en programación y diseño de algoritmos

Distinguir entre proposiciones simples y compuestas es esencial en la programación porque las estructuras de control (condicionales, bucles) se fundamentan directamente en expresiones booleanas que equivalen a proposiciones lógicas. Un programador que no diferencia estas categorías puede cometer errores lógicos graves en la evaluación de condiciones, produciendo comportamientos inesperados en el software [1].

Desde la perspectiva del diseño de algoritmos, las proposiciones compuestas permiten modelar condiciones complejas del mundo real que difícilmente pueden capturarse con una sola condición simple. La claridad en esta distinción facilita la escritura de código más legible, mantenible y con menor probabilidad de errores [1].

Ejemplos concretos en lenguajes de programación

1. Python – Proposición compuesta con conjunción ($\wedge$):

```
edad = 20 tiene_credencial = True # p: edad >= 18 | q: tiene_credencial #  
Proposicion compuesta: p AND q if edad >= 18 and tiene_credencial:  
print("Acceso permitido")
```

Aquí `edad >= 18` y `tiene_credencial` son proposiciones simples; su conjunción forma una proposición compuesta que determina el flujo del programa.

2. Java – Proposición compuesta con disyunción ($\vee$) y negación ($\neg$):

```
boolean esAdmin = false; boolean tieneSupervisor = true; // Compuesta: esAdmin  
OR tieneSupervisor if (esAdmin || tieneSupervisor) {  
System.out.println("Puede ver reportes"); }
```

3. C – Proposición condicional ($\rightarrow$) en validación de datos:

```
int x = -5; // Si x > 0 entonces aplicar raiz cuadrada // Equivale a: p -> q  
if (x > 0) { double raiz = sqrt(x); }
```

En este caso, no distinguir la proposición simple `x > 0` del compuesto general podría llevar a evaluaciones erróneas de condiciones más complejas como `x > 0 && x < 100`, que implica dos proposiciones simples unidas en conjunción. Un error en el diseño lógico de estas condiciones puede producir fallas de seguridad, bucles infinitos o resultados matemáticamente incorrectos [1].



7. Declaración de uso responsable de Inteligencia Artificial IA

El presente trabajo fue desarrollado de manera colaborativa y honesta por los estudiantes del presente grupo, en cumplimiento de los principios de transparencia, responsabilidad y honestidad académica promovidos por la asignatura.

Se declara expresamente que el uso de herramientas de Inteligencia Artificial se empleó exclusivamente como apoyo técnico para la generación de los ejemplos de código en los lenguajes Python, Java y C, incluidos tanto en la sección de respuesta a la pregunta de control como en la parte de resultados. Esta decisión se fundamenta en que, al encontrarnos en el primer ciclo (1A) de la carrera, aún estamos en proceso de adquirir el conocimiento técnico y las habilidades de programación necesarias para elaborar dichos ejemplos de forma totalmente autónoma.

El resto del contenido del documento, incluyendo el desarrollo, las conclusiones, recomendaciones, reflexión crítica y referencias bibliográficas, fue redactado y elaborado íntegramente por los integrantes del grupo.

Los autores asumimos plena responsabilidad sobre el contenido de este trabajo y nos comprometemos a utilizar las herramientas de IA de forma ética, crítica y transparente en nuestra formación académica y profesional.

8. Conclusiones

Se concluye que:

- La lógica proposicional constituye el fundamento matemático sobre el que se construye la programación moderna. Comprender la diferencia entre proposiciones simples y compuestas permite a los desarrolladores formular condiciones correctas en estructuras de control, reduciendo significativamente los errores lógicos en el código.
- Los conectores lógicos son el equivalente formal de los operadores booleanos presentes en todos los lenguajes de programación de alto nivel, lo que evidencia la relación directa entre la lógica matemática y la ingeniería de software.



unl

Universidad
Nacional
de Loja

9. Recomendaciones

Se recomienda que:

- Se recomienda complementar el estudio teórico de la lógica proposicional con la práctica sistemática de resolución de problemas en entornos reales de programación. Ejercicios que traduzcan expresiones lógicas formales directamente a código en Python, Java o C refuerzan la comprensión abstracta y desarrollan la capacidad de depuración lógica.
- Se recomienda a los estudiantes desarrollar el hábito de verificar manualmente sus expresiones lógicas mediante tablas de verdad antes de implementarlas en código, especialmente cuando se trabaja con condiciones compuestas de tres o más variables. Esta práctica preventiva evita errores sutiles de cortocircuito lógico —frecuentes en operadores AND y OR de lenguajes como Python o Java— y fomenta un razonamiento más estructurado y metódico.

10. Reflexión crítica

La lógica matemática es el lenguaje silencioso que rige cada decisión de un programa, desde una simple instrucción, hasta validaciones de seguridad complejas; por ello, ignorar sus fundamentos equivale a construir sobre arena. El verdadero reto académico no reside en la manipulación mecánica de tablas de verdad, sino en la capacidad de conectar la teoría con la práctica, comprendiendo que un operador mal posicionado no es solo un error académico, sino un fallo crítico que puede comprometer la integridad de un sistema o generar bucles infinitos en el flujo de ejecución.

11. Bibliografía / Referencias

[1] O. Levin, Discrete Mathematics: An Open Introduction, 4th ed. University of Northern Colorado, 2022. [En línea]. Disponible en: <https://discrete.openmathbooks.org>

12. Anexos

Tyrone Encarnación

DIÁ _____ MES _____ AÑO _____ NOTA 7

APE 1 FASE #2

Nombre del estudiante: Tyrone Encarnación

1. Ejercicios realizados en clase

Ejercicio 1

Premisas iniciales:

$$p \vee q = P_1$$
$$\neg r = P_2$$
$$q \rightarrow r = P_3$$

Desarrollo de Inferencias.

Premisa 4: $\neg q$

Premisa 4 obtenida mediante Modus Tollendo Tollens entre la premisa 2 ($\neg r$) y la premisa 3 ($q \rightarrow r$).

Premisa 5: p

Premisa 5 obtenida Modus Tollendo Ponens entre la premisa 1 ($p \vee q$) y la premisa 4 ($\neg q$).

Ejercicio 2

$$p \wedge p \leftrightarrow p$$

Simplificación

$$p \rightarrow p \wedge (p \vee \neg p)$$
$$p \rightarrow (p \wedge p) \vee (p \wedge \neg p)$$
$$p \rightarrow (p \wedge p) \vee (F)$$
$$p \rightarrow p \wedge p$$
$$p \rightarrow p$$

Leyes usadas

- Ley del Complemento
- Ley distributiva
- Ley de Complemento
- Ley de Idempotencia
- Ley de identidad

DIA MES AÑO
 NOTA

2. Simplificaciones de expresiones lógicas

Ejercicio 1

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)] \equiv (q \vee r)$$

Simplificación

Leyes usadas

$$[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$$

Ley de Morgan

$$\vee \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$$

Ley de Complemento

$$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)$$

Ley de identidad

$$[q \wedge (r \vee \neg r)] \vee [\neg q \wedge (r \vee \neg r)]$$

Ley distributiva

$$(q \wedge \vee) \vee (\neg q \wedge \vee)$$

Ley de Complemento

$$q \vee (\neg q \vee r)$$

Ley de identidad

$$(q \wedge \neg q) \wedge (q \vee r)$$

Ley distributiva

$$\vee \wedge (q \vee r)$$

Ley del complemento

$$q \vee r$$

Ley de identidad

Ejercicio 2

$$\neg(p \wedge q) \vee [(p \vee q) \wedge (\neg p \vee \neg q) \wedge (\neg p \wedge \neg q)]$$

Simplificación

$$(\neg p \vee \neg q) \vee [(p \vee q) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee \neg q)]$$

$$(\neg p \vee \neg q) \vee [(p \vee (q \wedge \neg q)) \wedge (\neg p \vee \neg q)]$$

$$(\neg p \vee \neg q) \vee [(p \vee F) \wedge (\neg p \vee \neg q)]$$

$$(\neg p \vee \neg q) \vee [p \wedge (\neg p \vee \neg q)]$$

$$(\neg p \vee \neg q) \vee [(p \wedge \neg p) \vee (p \wedge \neg q)]$$

$$(\neg p \vee \neg q) \vee [F \vee (p \wedge \neg q)]$$

$$(\neg p \vee \neg q) \vee (p \wedge \neg q)$$

$$(\neg p \vee (p \wedge \neg q)) \vee (\neg q \vee (p \wedge \neg q))$$

$$(\neg p \vee \neg q) \vee (\neg q \vee (p \wedge \neg q))$$

$$(\neg p \vee \neg q) \vee \neg q$$

$$\neg p \vee \neg q \vee \neg q$$

Leyes usadas

Ley de Morgan

Ley Distributiva

Ley de Complemento

Ley de identidad

Ley distributiva

Ley de complemento

Ley de identidad

Ley asociativa + conmutativa

Ley de Absorción

Ley de Absorción

Ley asociativa + conmutativa

DÍA MES AÑO NOTA

Ejercicio 3

$$[(\neg p \vee \neg q \vee \neg r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Simplificación

Leyes Usadas

$$[(p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Ley de Morgan

$$[p \wedge (q \wedge r) \vee (q \wedge \neg r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Ley de complemento

$$[p \wedge r] \vee [p \wedge \neg r] \vee [\neg p \wedge r]$$

Ley de identidad

$$[p \wedge (r \vee \neg r)] \vee (\neg p \wedge r)$$

Ley distributiva

$$[p \wedge r] \vee (\neg p \wedge r)$$

Ley de complemento

$$p \vee (\neg p \wedge r)$$

Ley de identidad

$$(p \vee \neg p) \wedge (p \vee r)$$

Ley distributiva

$$V \wedge (p \vee r)$$

Ley del complemento

$$p \vee r$$

Ley de identidad

3. Cómo las expresiones booleanas se aplican en la evaluación de condicionales en programación.

3.1 If-else

Ejemplo

Problema 1. Programa de aprobación de estudiantes

Desarrollar un programa que determine si un estudiante aprueba o reprueba una materia según las siguientes condiciones:

• El estudiante aprueba si:

• Tiene una nota alta (mayor o igual a 7).

O

• Tiene una nota media (mayor o igual a 5) y además entregó proyecto.

DIA: MES: AÑO:
 NOTA:

final.

En caso contrario, el estudiante reprueba.

Definición de variables.

- p: el estudiante tiene nota ≥ 7 (nota alta)
- r: el estudiante tiene nota ≥ 5 (nota media)
- q: el estudiante entregó el proyecto

Expresión lógica del problema

$$p \vee (r \wedge q)$$

Tabla de verdad

p	r	q	$r \wedge q$	$p \vee (r \wedge q)$	Resultado
V	V	V	V	V	Aproba
V	V	F	F	V	Aproba
V	F	V	F	V	Aproba
V	F	F	F	V	Aproba
F	V	V	V	V	Aproba
F	V	F	F	F	Reprueba
F	F	V	F	F	Reprueba
F	F	F	F	F	Reprueba



Algoritmo en Java

```
import java.util.Scanner;
```

```
public class AprobacionEstudiante { public  
static void main (String [] args) {
```

```
Scanner sc = new Scanner (System.in);
```

```
System.out.print("Ingrese la nota: "); int  
nota = sc.nextInt();
```

```
System.out.print("¿Entregó el proyecto? (true/false): "); boolean  
q = sc.nextBoolean();
```

```
boolean p = (nota >= 7); //nota alta
```

```
boolean r = (nota >= 5); //nota media
```

```
if (p || (r && q)) {
```

```
System.out.println("Aprobado");
```

```
} else {
```

```
System.out.println("Reprobado");
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```


Ejemplo de ejecución

Caso 1

- Nota = 6
 - Proyecto = entregado
- $p = F, r = V, q = V$
- Aprueba

Caso 2

- Nota = 4
 - Proyecto = no entregado
- $p = F, r = F, q = F$
- Resultado = F

El programa muestra:
Reprueba.

3.2 While

Ejemplos:

Problema 1: Control de Temperatura de un Reactor

Se requiere un sistema que mantenga un reactor funcionando mientras las condiciones de seguridad se cumplan. El reactor se detiene (sale del while) si la situación se vuelve peligrosa.

Condiciones para que el reactor siga operando:

- La temperatura es menor a 100°C (p).
- Y (el sistema de enfriamiento está activo (q) O el nivel de presión es normal (r)).



DIA MES AÑO NOTA

1. Definición de variables

- p: La temperatura es < 100 (Seguro).
- q: El enfriamiento está encendido.
- r: La presión es normal.

2. Expresión lógica del problema (Condición de continuación).

$p \wedge (q \vee r)$

Tabla de verdad

p	q	r	$(q \vee r)$	$p \wedge (q \vee r)$	Estado del Ciclo
V	V	V	V	V	Continúa
V	V	F	V	V	Continúa
V	F	V	V	V	Continúa
V	F	F	F	F	Termina
F	V	V	V	F	Termina
F	V	F	V	F	Termina
F	F	V	V	F	Termina
F	F	F	F	F	Termina

DIA	MES	AÑO	NOTA
-----	-----	-----	------

3. Algoritmo en Java

```
import java.util.Scanner;
```

```
public class ControlReactor {  
    public static void main(String[] args)  
    {  
        Scanner sc = new  
            Scanner(System.in);
```

```
        // Estado inicial para entrar al  
        ciclo boolean enfriamiento = true;
```

```
        boolean presionNormal = true;
```

```
        int temperatura = 20;
```

```
        while (temperatura < 100 CC (enfriamiento || presionNormal)) {
```

```
            System.out.println("Reactor estable. Temperatura actual:  
            " + temperatura + "°C");
```

```
            System.out.print("Ingrese nueva temperatura"); temperatura =  
            sc.nextInt();
```

```
            System.out.print("¿Enfriamiento activo? (true/false):");
```

```
            enfriamiento = sc.nextBoolean();
```

```
            System.out.print("¿Presión normal? (true/false):");
```

```
            presionNormal = sc.nextBoolean();
```

```
        }
```

```
        System.out.println("ALERTA: Condiciones inseguras detectadas. Ciclo terminado");  
        sc.close();
```

```
    }
```

3.3 Operadores lógicos en Python/C/Java

Ejemplos:

Problema 1: Acceso a Beca Universitaria

Premisas:

p: El promedio es mayor a 9.0.

q: El estudiante tiene excelencia deportiva.

Condición lógica: Se obtiene la beca si ocurre p o q.

(Disyunción: $p \vee q$).Tabla de Verdad ($p \vee q$)

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Código en Python

Promedio_alto = True # p

Excelencia_deportiva = False # q

If promedio_alto or excelencia_deportiva:

Print("Beca concedida.")

Else ("No cumple los requisitos para la beca.")

Jhoan Rojas

Matemáticas Discretas APE1 FASE 2

Resultados

1- Ejercicios realizados en clase

Ejercicio 1

Premisas iniciales:

$$p \vee q = P_1$$

$$\neg r = P_2$$

$$q \rightarrow r = P_3$$

Desarrollo de inferencias

Premisa 4: $\neg q$

Premisa 4 obtenida mediante Modus Tollendo Tollens entre premisa 2 ($\neg r$) y la premisa 3 ($q \rightarrow r$)

Premisa 5: p

Premisa 5 obtenida mediante Modus Tollendo Tollens entre la premisa 1 ($p \vee q$) y la premisa 4 ($\neg q$)

Ejercicio 2

$p \wedge p \leftrightarrow p$	
Simplificación	Leyes usadas
$p \rightarrow p \wedge (p \vee \neg p)$	Ley del complemento
$p \rightarrow (p \wedge p) \vee (p \wedge \neg p)$	Ley distributiva
$p \rightarrow (p \wedge p) \vee F$	Ley del complemento
$p \rightarrow p \wedge p$	Ley de idempotencia
$p \rightarrow p$	Ley de identidad

2- Simplificación de expresiones lógicas

Ejercicio 1

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)]$$

$$\rightarrow (q \vee r)$$

$$[(p \vee q \vee r) \vee (\neg p \vee \neg q \vee \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow (q \vee r)$$

Se aplica ley de Morgan

$$\vee \neg [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow (q \vee r) \text{ ley de complemento}$$

$$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \text{ ley de identidad}$$



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

$[q \wedge (r \vee r)] \vee (r \wedge r)$ Ley distributiva
 $(q \wedge \vee) \vee (r \wedge r)$ Ley del complemento
 $q \vee (r \wedge r)$ Ley de identidad
 $(q \vee q) \wedge (q \vee r)$ Ley distributiva
 $\vee \wedge (q \vee r)$ Ley de complemento
 $(q \vee r) \rightarrow (q \vee r)$ Ley de identidad

Ejercicio 2

$\neg (0 \wedge q) \vee [\neg p \vee q \wedge (\neg p \vee q) \wedge (r \vee r)]$
 $(\neg p \vee q) \vee [\neg p \vee q \wedge (\neg p \vee q) \wedge (r \vee r)]$ Ley de Morgan
 $(\neg p \vee q) \vee [\neg p \vee (q \wedge q) \wedge (r \vee r)]$ Ley distributiva
 $(\neg p \vee q) \vee [\neg p \vee F \wedge (r \vee r)]$ Ley de complemento
 $(\neg p \vee q) \vee [\neg p \wedge (r \vee r)]$ Ley de identidad
 $(\neg p \vee q) \vee [\neg p \wedge (r \vee r)]$ Ley distributiva
 $(\neg p \vee q) \vee [F \vee (p \vee r)]$ Ley de complemento
 $(\neg p \vee q) \vee (p \vee r)$ Ley de identidad
 $[\neg p \vee (p \wedge r)] \vee [q \vee (p \wedge r)]$ Ley asociativa + conmutativa
 $(\neg p \vee r) \vee [q \vee (p \wedge r)]$ Ley de Absorción
 $(\neg p \vee r) \vee q$ Ley de Absorción
 $\neg p \vee q \vee r$ Ley asociativa + conmutativa

Ejercicio 3

$[(\neg p \vee q \vee r) \vee (p \wedge q \wedge r)] \wedge [(\neg p \vee r) \vee (p \wedge r) \vee (q \wedge r)]$
 $[(\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)] \wedge [(\neg p \vee r) \vee (p \wedge r) \vee (q \wedge r)]$
 Ley de Morgan
 $\vee \wedge [(\neg p \vee r) \vee (p \wedge r) \vee (q \wedge r)]$ Ley complemento

$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \wedge r)]$	Ley de Identidad
$[p \wedge (r \vee \neg r) \vee (\neg p \wedge r)]$	Ley distributiva
$(p \wedge \neg p) \vee (\neg p \wedge r)$	Ley del complemento
$\neg p \vee (\neg p \wedge r)$	Ley de identidad
$(p \vee \neg p) \wedge (p \vee r)$	Ley distributiva
$\neg p \vee (p \vee r)$	Ley del complemento
$p \vee \neg p$	Ley de identidad

3.0 ¿Cómo las expresiones booleanas se aplican a la evaluación de condiciones en programación?

3.1 IF - else

Ejemplo

Problema: Se desea desarrollar un programa que determine si un estudiante aprueba o reprueba una materia según las siguientes condiciones:

- El estudiante aprueba si:
 - Tiene una nota alta (mayor o igual a 7)
 - Tiene una nota media (mayor o igual a 5) y además entregó el proyecto final.
- En caso contrario, el estudiante reprueba.

Variables

p : el estudiante tiene nota ≥ 7

r : el estudiante tiene nota ≥ 5

q : el estudiante entregó el proyecto

$$p \vee (r \wedge q)$$

Tabla de verdad

p	r	q	$r \wedge q$	$p \vee (r \wedge q)$	Resultado
V	V	V	V	V	Aprobado
V	V	F	F	V	Aprobado
V	F	V	F	V	Aprobado
V	F	F	F	V	Aprobado
F	V	V	V	V	Aprobado
F	V	F	F	F	Reprobado
F	F	V	F	F	Reprobado
F	F	F	F	F	Reprobado

Algoritmo en Java

```
import java.util.Scanner;

public class AprobaciónEstudiante
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese la nota: ");
        int nota = sc.nextInt();

        System.out.print("¿Entregó el proyecto? (true/false): ");
        boolean q = sc.nextBoolean();

        boolean p = (nota >= 7); // nota alta
        boolean r = (nota >= 5); // nota media

        if (p || (r && q))
        {
            System.out.println("Aprobado");
        }
        else
        {
            System.out.println("Reprobado");
        }
        sc.close();
    }
}
```


Prueba de escritorio

Caso 1

- Nota = 6
- Proyecto = entregado
- $p = F, r = V, q = V$

Aprueba

Caso 2

- Nota = 4
- Proyecto = No entregado
- $p = F, r = F, q = F$

Reprueba

3.2 While

Problema 1

Se requiere un sistema que mantenga un reactor funcionando mientras las condiciones de seguridad se cumplan. El reactor se detiene (sale del while) si la situación se vuelve peligrosa.

Condiciones para que el reactor siga operando

- La temperatura es menor a 100°C (p)
- Y (el sistema de enfriamiento este activo (q) O el nivel de presión es normal (r))

Variables

p : la temperatura es < 100

q : Enfriamiento encendido

r : la presión es normal

$$p \wedge (q \vee r)$$

Tabla de verdad

p	q	r	$(q \vee r)$	$p \wedge (q \vee r)$	Estado
V	V	V	V	V	Continúa
V	V	F	V	V	Continúa
V	F	V	V	V	Continúa
V	F	F	F	F	Termina
F	V	V	V	F	Termina
F	V	F	F	F	Termina
F	F	V	V	F	Termina
F	F	F	F	F	Termina

Algoritmo en Java

```
import java.util.Scanner;
```

```
public class ControlReactor {
```

```
    public static void main (String [] args) {  
        Scanner sc = new Scanner (System.in);
```

```
        // Estado inicial para entrar al ciclo boolean
```

```
        enfriamiento = true;  
        boolean presionNormal = true;  
        int temperatura = 20;
```

```
        While (temperatura < 100 CC (enfriamiento || presionNormal)) {  
            System.out.println ("Reactor estable. Temperatura actual: +  
            temperatura + "°C");
```

```
            System.out.print ("Ingrese nueva temperatura ");  
            temperatura = sc.nextInt ();
```

```
            System.out.print ("¿Enfriamiento activo? (true/false) ");  
            enfriamiento = sc.nextBoolean ();
```

```
            System.out.print ("Presión normal? (true/false) ");  
            presionNormal = sc.nextBoolean ();
```

```
        }
```

```
        System.out.print ("Alerta: condiciones inseguras. Ciclo termi-  
        nado.");  
        sc.close ();
```

```
    }
```

3.3 Operadores lógicos

Problema: Acceso a Beca Universitaria

Premisas:

p. El promedio es mayor a 9.0

q. El estudiante tiene excelencia deportiva

 $(p \vee q)$

Tabla de verdad

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Código en python

Promedio_alto = True # p

Excelencia_deportiva = False # q

```
if promedio_alto or excelencia_deportiva:
    print("Beca concedida")
```

```
else:
    print("No cumple los requisitos para la beca.")
```



Erick Andrade

DÍA

MES

AÑO

Matemáticas discretas APEI Fase 2

NOTA

1. Ejercicios realizados en clase

Ejercicio 1

Premisas Iniciales:

$$p \vee q = P_1$$

$$\neg r = P_2$$

$$Q \rightarrow r = P_3$$

Desarrollo de Inferencias:

Premisa 4: $\neg q$

Premisa 4 obtenida mediante Modus Tollendo Tollens entre la premisa 2 ($\neg r$) y la premisa 3 ($q \rightarrow r$).

Premisa 5: p

Premisa 5 obtenida mediante Modus Tollendo Ponens entre la premisa 1 ($p \vee q$) y la premisa 4 ($\neg q$).

Ejercicio 2

$p \wedge p \leftrightarrow p$

Simplificación

$$p \rightarrow p \wedge (p \vee \neg p)$$

$$p \rightarrow (p \wedge p) \vee (p \wedge \neg p)$$

$$p \rightarrow (p \wedge p) \vee (F)$$

$$p \rightarrow p \wedge p$$

$$p \rightarrow p$$

Leyes usadas

Ley de Complemento

Ley distributiva

Ley de Complemento

Ley de Idempotencia

Ley de Identiadad

2. Simplificación de expresiones lógicas

Ejercicio 1.

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)] = (q \vee r)$$

Simplificación

Leyes usadas

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \vee \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$$

Ley de Morgan

$$V \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$$

Ley de Complemento

$$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)$$

Ley de identidad

$$[q \wedge (r \vee \neg r)] \vee [\neg q \wedge (r \vee \neg r)]$$

Ley distributiva

$$(q \wedge V) \vee (\neg q \wedge V)$$

Ley del complemento

$$q \vee (\neg q \wedge V)$$

Ley de identidad

$$(q \vee \neg q) \wedge (q \vee V)$$

Ley distributiva

$$V \wedge (q \vee V)$$

Ley de complemento

$$q \vee V$$

Ley de identidad

Ejercicio 2

$$\neg(q \wedge q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \wedge r)]$$

Simplificación

Leyes usadas



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

$$(\neg p \vee \neg q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee r)]$$

Ley de Morgan

$$\neg (p \vee \neg q) \vee [(p \vee (q \wedge \neg q)) \wedge (\neg p \vee r)]$$

Ley distributiva

$$(\neg p \vee \neg q) \vee [(p \vee F) \wedge (\neg p \vee r)]$$

Ley de complemento

$$[\neg p \vee \neg q] \vee [p \wedge (\neg p \vee r)]$$

Ley de identidad

$$(\neg p \vee \neg q) \vee [(p \wedge \neg p) \vee (p \wedge r)]$$

Ley distributiva

$$(\neg p \vee \neg q) \vee [F \vee (p \wedge r)]$$

Ley de complemento

$$(\neg p \vee \neg q) \vee [p \wedge r]$$

Ley de identidad

$$(\neg p \vee (p \wedge r)) \vee (\neg q \vee (p \wedge r))$$

Ley asociativa + conmutativa

$$(\neg p \vee r) \vee (\neg q \vee (p \wedge r))$$

Ley de absorción

$$(\neg p \vee r) \vee \neg q$$

Ley de absorción

$$\neg p \vee \neg q \vee r$$

Ley asociativa + conmutativa



Ejercicio 2

$$[(\neg p \vee \neg q \vee \neg r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Simplificación

Leyes usadas

$$[(\neg p \wedge q \vee r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Ley de Morgan

$$\vee \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Ley de complemento

$$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$$

Ley de identidad

$$[p \wedge (r \vee \neg r)] \vee (\neg p \vee r)$$

Ley distributiva

$$(p \wedge \vee) \vee (\neg p \vee r)$$

Ley de complemento

$$p \vee (\neg p \vee r)$$

Ley de identidad

$$(p \vee \neg p) \wedge (p \vee r)$$

Ley del distributiva

$$\vee \wedge (p \vee r)$$

Ley del complemento

$$p \vee r$$

Ley de identidad

If - else:

Problema 1: Programa de aprobación de estudiante.

Se desea desarrollar un programa que determine si un estudiante aprueba o reprueba una materia según las siguientes condiciones:

El estudiante aprueba si:

Tiene una nota alta (mayor o igual a 7),
o

Tiene una nota media (mayor o igual a 5) y además entregó el proyecto final.

En caso contrario, el estudiante reprueba.

Definición de variables:

p: el estudiante tiene nota ≥ 7 (nota alta)

r: el estudiante tiene nota ≥ 5 (nota media)

q: el estudiante entregó el proyecto

Expresión lógica del problema

$PV(r \wedge q)$

Tabla de verdad

p	r	q	$r \wedge q$	$PV(r \wedge q)$	Resultado
V	V	V	V	V	Aprueba
V	V	F	F	V	Aprueba
V	F	V	F	V	Aprueba
V	F	F	F	V	Aprueba
F	V	V	V	V	Aprueba
F	V	F	F	F	Reprueba
F	F	V	F	F	Reprueba
F	F	F	F	F	Reprueba

Algoritmo en Java:

```
import java.util.Scanner;
```

```
public class AprobacionEstudiante {public
```

```
static void main(String[] args){
```

```
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Ingrese la nota: "); int
```

```
nota = sc.nextInt();
```

```
System.out.print("¿Entrega el proyecto (true/false): "); boolean q =
```

```
sc.nextBoolean();
```

```
boolean p = (nota >= 7); // nota alta
```

```
boolean r = (nota >= 5); // nota media
```

```
if (p || (r && q)) {
```

```
System.out.println("Aprobado");
```

```
} else {
```

```
System.out.println("Reprueba");
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Ejemplo de ejecución:

Caso 1:

Nota = 6

Proyecto = entregado

$p = F, r = V, q = V$

→ Aprueba

Caso 2

Nota = 4

Proyecto = no entregado

$p = F, r = F, q = F$

Resultado = F

El programa muestra: Reprocha

While

Ejemplo: Control de temperatura de un reactor

Se requiere un sistema que mantenga un reactor funcionando mientras las condiciones de seguridad se cumplen. El reactor se detiene (sale del ciclo) si la situación se vuelve peligrosa.

*Condiciones para que el reactor siga operando.

- La temperatura es menor a 100°C (p).
- Y (el sistema de enfriamiento está activo (q) y el nivel de presión es normal (r)).

Definición de variables

- p: La temperatura es < 100 (seguro).
- q: El enfriamiento está encendido.
- r: La presión es normal.

Expresión lógica del problema

$$p \wedge (q \vee r)$$

Tablas de verdad

p	q	r	$(q \vee r)$	$p \wedge (q \vee r)$	Estado
V	V	V	V	V	continúa
V	V	F	V	V	continúa
V	F	V	V	V	continúa
V	F	F	F	F	termina
F	V	V	V	F	termina
F	V	F	V	F	termina
F	F	V	V	F	termina
F	F	F	F	F	termina

Algoritmo en Java

```
import java.util.Scanner;

public class ControlReactor {
    public static void main (String[] args) {
        Scanner sc = new
        Scanner (System.in);

        // Estado inicial para entrar al ciclo
        boolean enfriamiento = true;
        boolean presionNormal = true;
        int temperatura = 20;

        while (temperatura < 100 && (enfriamiento && presionNormal)) {
            System.out.println("Reactor estable. Temperatura actual: " + temperatura + "°C");

            System.out.print("Ingrese nueva temperatura: ");
            temperatura = sc.nextInt();

            System.out.print("¿Enfriamiento activo? (true/false): ");
            enfriamiento = sc.nextBoolean();

            System.out.print("¿Presión normal? (true/false): ");
            presionNormal = sc.nextBoolean();
        }

        System.out.println("¡Alerta! condiciones inseguras de techadas. Ciclo terminado.");
        sc.close();
    }
}
```

Operadores lógicos en Python/C/Java

Ejemplo:

Problema 1: Acceso a Beca Universitaria

Premisas:

p: El promedio es mayor a 9.0.

q: El estudiante tiene excelencia deportiva

Condición lógica: Se obtiene la beca si ocurre $p \vee q$.

(Disyunción: $p \vee q$)

Tabla de Verdad ($p \vee q$)

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Código en Python

Promedio_alto = True $\wedge$ p

Excelencia_deportiva = False $\wedge$ q

```
If promedio_alto or excelencia_deportiva:  
    print("Beca concedida.")
```

```
else:  
    print("No cumple los requisitos para la beca.")
```



Matias Ortega

1. Resultados

Matemáticas Discretas APE 1 FASE 2

1. Ejercicios realizados en clase

Ejercicio 1

Premisas Iniciales:

$$p \vee q = p_1$$

$$\neg r = p_2$$

$$q \rightarrow r = p_3$$

Desarrollo de Inferencias:

Premisa 4: $\neg q$

Premisa 4 obtenida mediante Modus Tollendo Tollens entre la premisa 2 ($\neg r$) y la premisa 3 ($q \rightarrow r$).

Premisa 5: p

Premisa 5 obtenida mediante Modus Tollendo Ponens entre la premisa 1 ($p \vee q$) y la premisa 4 ($\neg q$).

Ejercicio 2

$p \wedge p \Leftrightarrow p$	
Simplificación	Leyes usadas
$p \rightarrow p \wedge (p \vee \neg p)$	Ley de Complemento
$p \rightarrow (p \wedge p) \vee (p \wedge \neg p)$	Ley distributiva
$p \rightarrow (p \wedge p) \vee (F)$	Ley de Complemento
$p \rightarrow p \wedge p$	Ley de Idempotencia
$p \rightarrow p$	Ley de identidad

2. Simplificación de expresiones lógicas

Ejercicio 1

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)] \equiv (q \vee r)$$

Simplificación	2 leyes usadas
$[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]]$	2 ley de Morgan
$\vee \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)]$	2 ley de Complemento
$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r) \vee (\neg q \wedge \neg r)$	2 ley de identidad
$[q \wedge (r \vee \neg r)] \vee [\neg q \wedge (r \vee \neg r)]$	2 ley distributiva
$(q \wedge \vee) \vee (\neg q \wedge \vee)$	2 ley Complemento
$q \vee (\neg q \vee r)$	2 ley identidad
$(q \wedge \neg q) \wedge (q \vee r)$	2 ley distributiva
$\vee \wedge (q \vee r)$	2 ley Complemento
$q \vee r$	2 ley identidad

Ejercicio 2

$$\neg(p \wedge q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \wedge r)]$$

Simplificación	2 leyes usadas
$(\neg p \vee \neg q) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee r)]$	2 ley de Morgan
$(\neg p \vee \neg q) \vee [(p \vee (q \wedge \neg q)) \wedge (\neg p \vee r)]$	2 ley Distributiva
$(\neg p \vee \neg q) \vee [(p \vee \vee) \wedge (\neg p \vee r)]$	2 ley Complemento
$(\neg p \vee \neg q) \vee [p \wedge (\neg p \vee r)]$	2 ley Identidad
$[(\neg p \vee \neg q) \vee (p \wedge \neg p) \vee (p \wedge r)]$	2 ley Distributiva



$(\neg p \vee \neg q) \vee (F \vee (p \wedge r))$	Ley Complemento
$(\neg p \vee \neg q) \vee (p \vee r)$	Ley Identidad
$(\neg p \vee (p \wedge r)) \vee (\neg q \vee (p \wedge r))$	Ley Asociativa + Comutativa
$(\neg p \vee r) \vee (\neg q \vee (p \wedge r))$	Ley Absorción
$(\neg p \vee r) \vee \neg q$	Ley Absorción
$\neg p \vee \neg q \vee r$	Ley Asociativa + Comutativa

Ejercicio 3

$[(\neg p \vee \neg q \vee \neg r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	
Simplificación	Leyes de Morgan
$[\neg(p \wedge q \vee r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley de Morgan
$[\neg(p \wedge q \vee r) \vee (p \wedge q \wedge r)] \wedge [(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley Complemento
$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley Identidad
$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley distributiva
$[(p \wedge r) \vee (p \wedge \neg r) \vee (\neg p \vee r)]$	Ley complemento
$(p \wedge r) \vee (\neg p \vee r)$	Ley identidad
$p \vee (\neg p \vee r)$	Ley distributiva
$(p \vee \neg p) \wedge (p \vee r)$	Ley complemento
$1 \wedge (p \vee r)$	Ley identidad
$p \vee r$	

3: Cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación.

3.4 If-else

Ejemplo:

Problema 1. Programa de aprobación de estudiantes

Se desea desarrollar un programa que determine si un estudiante *aproba* o *reproba* una materia según las siguientes condiciones:

- El estudiante *aproba* si:
 - Tiene una nota alta (mayor o igual a 7),
 - Tiene una nota media (mayor o igual a 5) y además entregó el proyecto final

En caso contrario, el estudiante *reproba*.

Definición de variables

- p : el estudiante tiene nota ≥ 7 (nota alta)
- r : el estudiante tiene nota ≥ 5
- q : el estudiante entregó el proyecto

Expresión lógica del problema
 $p \vee (r \wedge q)$

Tabla de verdad					
p	r	q	$r \wedge q$	$p \vee (r \wedge q)$	Resultado
V	V	V	V	V	Aproba
V	V	F	F	V	Aproba
V	F	V	F	V	Aproba
V	F	F	F	V	Aproba
F	V	V	V	V	Aproba
F	V	F	F	F	Reproba
F	F	V	F	F	Reproba
F	F	F	F	F	Reproba

Algoritmo en Java

```
import java.util.Scanner;
```

```

public class AprobacionEstudiante { public
static void main (String args) {
Scanner sc = new Scanner (System.in);

System.out. print ("Ingrese la nota:"); int
nota = sc.nextInt();

System.out. print ("¿Entregó el proyecto? (true/false):"); boolean q =
sc.nextBoolean();

boolean p = (nota >= 7); // nota alta
boolean r = (nota >= 5); // nota media

if (p || (r & q)) {
System.out. println ("Aprobado");
} else {
System.out. println ("Reprobado");
}

sc.close();
}
}

```

Ejemplo de ejecución

Caso 1

- Nota = 6
- Proyecto = entregado
- p = F, r = V, q = V
- Aprobado

Caso 2

- Nota = 4
- Proyecto = no entregado
- p = F, r = F, q = F
- Resultado = F
- El programa muestra:
- Reprobado

3.2 While

Ejemplos:

Problema: Control de temperatura de un reactor

Se requiere un sistema que mantenga un reactor funcionando mientras las condiciones de seguridad se cumplan. El reactor se detiene (sale del while) si la situación se vuelve peligrosa.

Condiciones para que el reactor siga operando:
- q : 2a temperatura es menor a 100°C (segura)
- r : El sistema de enfriamiento esta encendido
- p : El nivel de presión es normal (r).

1. Definición de variables:
- p : 2a temperatura es ≤ 100 (segura)
- q : El enfriamiento esta encendido
- r : 2a presión es normal

2. Expresión lógica del problema (Condición de continuación)

$p \wedge (q \vee r)$

Tabla de Verdad

p	q	r	$(q \vee r)$	$p \wedge (q \vee r)$	Estado del ciclo
V	V	V	V	V	CONTINUA
V	V	F	V	V	CONTINUA
V	F	V	V	V	CONTINUA
V	F	F	F	F	TERMINA
F	V	V	V	F	TERMINA
F	V	F	V	F	TERMINA
F	F	V	V	F	TERMINA
F	F	F	F	F	TERMINA

3. Algoritmo en Java

```
import java.util.Scanner;
```

```
public class ControlReactor {
    public static void main (String args) {
        Scanner sc = new
        Scanner (System.in);
```

// Estado inicial para entrar al ciclo

```
boolean enfriamiento = true;
boolean presionNormal = true;
int temperatura = 20;
```

```
while (temperatura < 100 ((enfriamiento || presionNormal)) {
    System.out.println ("Reactor estable, temperatura actual: " + temperatura
    + "°C");
```

```
    System.out.print ("Ingrese nueva temperatura: "); temperatura
    = sc.nextInt();
    System.out.print ("¿Enfriamiento activo? (true/false): ");
```



```
enfriamiento = sc.nextBoolean()
System.out.println("Presion normal (" + true + false + ")",
presionNormal = sc.nextBoolean());
System.out.println("ALERTA: condiciones inseguras detectadas ciclo
terminado", sc.close());
```

3.3. Operadores lógicos en Python (C/Java)

Ejemplos:

Problema 1: Acceso a Beca Universitaria

Premisas:

p: El promedio es mayor a 5.0

q: El estudiante tiene excelencia deportiva

Condición lógica: se obtiene la beca si ocurre p o q. (Disyunción: p v q)

Tabla de Verdad (p v q)

p	q	p v q
V	V	V
V	F	V
F	V	V
F	F	F

Código en Python

```
Promedio-alto = True #p
Excelencia-deportiva = False
#q
```

```
If promedio-alto-o-excelencia-deportiva;
Print ("Beca concedida.")
```

```
Else:
Print ("No cumple los requisitos para la beca.")
```